

Worked Solutions

Data Structures

Set B: Original | CSE Semester 3

Companion to ds-original.pdf. Question numbers match that paper. Equivalent correct methods are acceptable. These are independent worked solutions, not an official university marking scheme.

Questions 1–5: MCQ answer key, 1 mark each (5 marks total). Questions 6 onward: theory solutions (25 marks total).

1. Structure definition

Correct option: (C). `struct` defines a type whose members can hold different kinds of data.

2. Nested member access

Correct option: (A). For structure objects, each member is accessed with a dot: `s.admitted.year`.

3. String capacity

Correct option: (D). One of the 40 positions must hold the terminating null character.

4. Safe input width

Correct option: (B). `%39s` reads at most 39 characters and appends the null terminator.

5. Pointer member access

Correct option: (C). `p->roll` is equivalent to `(*p).roll`.

6. Contiguous dynamic matrix

```
#include <stdlib.h>
int *makeMatrix(int r, int c) {
    if (r <= 0 || c <= 0) return NULL;
    /* The question guarantees that this size fits size_t. */
    return malloc((size_t)r * (size_t)c * sizeof(int));
}
/* Access (i,j) as *(base + (size_t)i * c + j).
Valid indices satisfy 0 <= i < r and 0 <= j < c.
Caller checks the returned pointer and eventually frees it. */
```

Rows are adjacent in memory, so row i starts at offset ic and column j adds j . Allocation requests $rc \text{ sizeof(int)}$ bytes. A non-null result must be initialized before its elements are read and released with `free` when no longer needed.

7. Dangling pointer and memory leak

A dangling pointer refers to storage whose lifetime has ended:

```
int *p = malloc(sizeof *p);
free(p);      /* p now dangles; do not dereference it. */
p = NULL;     /* Clear this pointer after releasing storage. */
```

Any aliases must also stop using that storage. A memory leak loses the final reference to allocated storage:

```
int *p = malloc(sizeof *p);
p = NULL;     /* Wrong: allocated storage can no longer be freed. */
```

Keep an owning pointer and free the allocation before overwriting that final reference. These examples illustrate errors; they are not code to copy into a working implementation.

8. Delete all matching nodes

The pointer link always addresses the pointer that leads to the current node. On deletion, update that pointer and keep link in place so the next node is checked. On retention, advance to the next link. This covers head, consecutive and all-node matches, and releases each removed allocation. Time is $O(n)$; auxiliary space is $O(1)$. The following shared listing also contains the reversal for 9.

```
#include <stdlib.h>
typedef struct Node {
    int value;
    struct Node *next;
} Node;
void deleteAll(Node **head, int key) {
    if (!head) return;
    Node **link = head;
    while (*link) {
        if ((*link)->value == key) {
            Node *old = *link;
            *link = old->next;
            free(old);
        } else link = &(*link)->next;
    }
}
Node *reverse(Node *head) {
    Node *prev = NULL, *cur = head;
    while (cur) {
        Node *next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    return prev;
}
```

9. Reverse a singly linked list

Use the reverse function in the listing for 8. Save the original successor before changing cur's next pointer; move prev and cur forward. The loop invariant is that prev heads the reversed processed prefix and cur heads the untouched suffix. At termination, prev is the new head. Assign `head = reverse(head)`. Empty and one-node lists work unchanged. Time is $O(n)$ and auxiliary space is $O(1)$.

10. List trace

$$\begin{array}{ccccccc}
 10 \rightarrow 20 \rightarrow 30 & \longrightarrow & 5 \rightarrow 10 \rightarrow 20 \rightarrow 30 \\
 \longrightarrow & 5 \rightarrow 10 \rightarrow 20 \rightarrow 25 \rightarrow 30 & \longrightarrow & \boxed{5 \rightarrow 10 \rightarrow 20 \rightarrow 25}.
 \end{array}$$

Starting at the final head, follow three next links to reach tail 25. Checking that the tail's next pointer is NULL does not traverse another node-to-node link.

11. Stack trace for infix conversion

The stack is listed bottom to top. Exponentiation is right-associative, so the second exponentiation does not pop the first.

Token	Operator stack	Output
A	empty	A
+	+	A
B	+	AB
*	+ *	AB
(	+ * (	AB
C	+ * (	ABC
-	+ * (-	ABC
D	+ * (-	ABCD
)	+ *	ABCD-
^	+ * ^	ABCD-
E	+ * ^	ABCD-E
^	+ * ^ ^	ABCD-E
F	+ * ^ ^	ABCD-EF
End	empty	ABCD-EF^^*+

Thus the final postfix form is `ABCD-EF^^*+`. The two exponentiations encode $(C - D)^{(E^F)}$.

12. Balanced brackets

Push each opening bracket. For a closing bracket, require a nonempty stack and matching opening type. Accept only if the stack is empty at the end. Time is $O(n)$; the fixed array uses capacity 100, as permitted by the question.

```
int balanced(const char *s) {
    char stack[100];
    int top = 0;
    for (; *s; ++s) {
        if (*s == '(' || *s == '[') {
            if (top == 100) return 0;
            stack[top++] = *s;
        } else if (*s == ')' || *s == ']') {
            if (!top) return 0;
            char open = stack[--top];
            if ((*s == ')') && open != '(' ||
                (*s == ']') && open != '[')) return 0;
        } else return 0;
    }
    return top == 0;
}
```

13. Undo/redo trace

Operation	Undo: bottom to top	Redo: bottom to top
Edits A, B, C	A, B, C	empty
Undo	A, B	C
Undo	A	C, B
Redo	A, B	C
New edit D	A, B, D	empty

Undo transfers the newest applied action to redo; redo reapplies its newest undone action. New edit D clears redo, since C belongs to the abandoned editing branch.